

Tree Metrics

Binary Trees, Height & Diameter · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Compute a tree's height and its diameter, both in edges. You are given starter code with two functions. The height helper is almost complete but has a bug, and the diameter function is an empty stub. Fix the bug and implement the stub.

What to Compute

- Height: the longest root-to-leaf path in edges (empty = -1, single node = 0).
- Diameter: the longest path between any two nodes, in edges, bending at any node.

Input / Output Format

Input: One line: the tree in level order (values and N).

Output: Two lines: Height, Diameter.

Examples

Example 1

Input: tree = 1 2 3 4 5 N 6 7

Output:

Height: 3
Diameter: 5

Explanation: The deepest path is three edges; the longest path between two nodes spans five edges.

Example 2

Input: tree = 1 2 3

Output:

Height: 1
Diameter: 2

Explanation: Two children give height 1 and a diameter of 2 (left-root-right).

Constraints

- Both are measured in edges.
- The diameter path may bend at any node and need not pass through the root.
- Solve the diameter in one post-order pass over the tree.

Your Task

- Task 1 - Fix the bug. The height helper is flagged with a comment, but the bug's location is not given. Find the single bug so an empty subtree contributes -1 (not 0).
- Task 2 - Implement the diameter from scratch: in one post-order pass, track the maximum of $\text{height}(\text{left}) + \text{height}(\text{right}) + 2$ over all nodes.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
# Lab T3 - Tree Metrics (height and diameter, both in edges)
import sys
from collections import deque
class Node:
    def __init__(self, v): self.v = v; self.left = None; self.right = None
def build(tokens):
    if not tokens or tokens[0] == "N": return None
    root = Node(int(tokens[0])); q = deque([root]); i = 1
    while q and i < len(tokens):
        node = q.popleft()
        if i < len(tokens):
            if tokens[i] != "N": node.left = Node(int(tokens[i])); q.append(node.left)
            i += 1
        if i < len(tokens):
            if tokens[i] != "N": node.right = Node(int(tokens[i])); q.append(node.right)
            i += 1
    return root
# This function contains a BUG. Find it and fix it.
# Height is measured in edges: an empty subtree is -1, a single node is 0.
def height(n):
    if not n: return 0
    return 1 + max(height(n.left), height(n.right))
# TODO: Implement this function from scratch.
# Return the diameter (longest path between any two nodes, in edges).
def diameter(n):
    return 0
def main():
    root = build(sys.stdin.read().split())
    print("Height: " + str(height(root)))
    print("Diameter: " + str(diameter(root)))
```

```
main()
```

C++

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <sstream>
#include <functional>
#include <algorithm>
using namespace std;
struct Node { long long v; Node* left; Node* right; Node(long long
x):v(x),left(nullptr),right(nullptr){ } };
Node* build(const vector<string>& t){
    if (t.empty() || t[0]=="N") return nullptr;
    Node* root=new Node(stoll(t[0])); queue<Node*> q; q.push(root); size_t i=1;
    while(!q.empty() && i<t.size()){ Node* node=q.front(); q.pop();
        if (i<t.size()){ if (t[i]!="N"){ node->left=new Node(stoll(t[i])); q.push(node->left);} i++; }
        if (i<t.size()){ if (t[i]!="N"){ node->right=new Node(stoll(t[i])); q.push(node->right);} i+
+; } }
    return root;
}
vector<string> readTokens(){ vector<string> t; string s; while(cin>>s) t.push_back(s); return t; }
// This function contains a BUG. Find it and fix it.
// Height is measured in edges: an empty subtree is -1, a single node is 0.
int heightF(Node* n){ return !n?0:1+max(heightF(n->left),heightF(n->right)); }
// TODO: Implement this function from scratch.
// Return the diameter (longest path between any two nodes, in edges).
long long diameter(Node* n){ return 0; }
int main(){ Node* r=build(readTokens()); cout<<"Height: "<<heightF(r)<<"\n"<<"Diameter:
"<<diameter(r)<<"\n"; return 0; }
```

Java

```
import java.util.*;
public class Main {
    static class Node { long v; Node left, right; Node(long x){v=x;} }
    static Node build(String[] t){ if (t.length==0||t[0].equals("N")) return null; Node root=new
Node(Long.parseLong(t[0])); Queue<Node> q=new LinkedList<>(); q.add(root); int i=1;
    while(!q.isEmpty()&&i<t.length){ Node n=q.poll(); if(i<t.length){ if(!t[i].equals("N"))
{ n.left=new Node(Long.parseLong(t[i])); q.add(n.left);} i++; } if(i<t.length){ if(!t[i].equals("N")){
n.right=new Node(Long.parseLong(t[i])); q.add(n.right);} i++; } } return root; }
    // This function contains a BUG. Find it and fix it.
    // Height is measured in edges: an empty subtree is -1, a single node is 0.
    static int heightF(Node n){ return n==null?0:1+Math.max(heightF(n.left),heightF(n.right)); }
    // TODO: Implement this function from scratch.
    // Return the diameter (longest path between any two nodes, in edges).
    static long diameter(Node n){ return 0; }
    public static void main(String[] a){ Scanner sc=new Scanner(System.in); List<String> t=new
ArrayList<>(); while(sc.hasNext()) t.add(sc.next());
    Node r=build(t.toArray(new String[0]));
    System.out.println("Height: "+heightF(r)); System.out.println("Diameter: "+diameter(r)); }
}
```

Sample Run

Sample Input

```
1 2 3 4 5 N 6 7
```

Sample Output

```
Height: 3
```

```
Diameter: 5
```